

FlowWave-AI: A Real-Time, Cloud-Native System for Intelligent Traffic Management and Dynamic Green Corridor Prioritization

1st Chandrashekhar
Mahajan
DESH
Pune ,India

chandrashekhar.mahajan@vit.edu

2nd Tanmay S.
Kumbhare
DESH
Pune ,India

tanmay.kumbhare24@vit.edu

3rd Nupur V. Lade
DESH
Pune ,India

nupur.lade24@vit.edu

4th Shripad P. Kulkarni
DESH
Pune , India

shripad.kulkarni24@vit.edu

5th Hemantkumar S.
Lakhane
DESH
Pune ,India

hemantkumar.lakhane24@vit.edu

**Department of Engineering, Sciences and Humanities (DESH)
Vishwakarma Institute of Technology, Pune, Maharashtra, India**

Abstract — Urban traffic congestion and delays faced by emergency vehicles pose critical challenges to modern cities. Conventional Traffic Management Systems (TMS) are typically static, unscalable, and lack real-time, multi-stakeholder data synchronization. This paper introduces FlowWave-AI, a cloud-native Intelligent Traffic Management System (ITMS) designed on a modern web architecture. The system employs React for its responsive frontend and Supabase as a backend-as-a-service (BaaS), offering a scalable PostgreSQL database, real-time data streaming, and serverless edge functions. These functions execute the core control logic, including a novel physics-informed Green Signal Time (GST) calculation formula that dynamically adapts signal durations based on real-time traffic and emergency vehicle conditions. The system’s primary innovation lies in its multi-role, three-dashboard framework: (1) an Authority Dashboard for centralized monitoring and manual control; (2) an Emergency Dashboard that enables drivers to request a “Dynamic Green Corridor” automatically prioritizing signals along their route; and (3) a Citizen Dashboard that enhances public transparency. The control logic was validated using SUMO simulations, demonstrating substantial performance improvements over conventional fixed-time controllers. FlowWave-AI provides a scalable, data-driven, and practical platform integrating real-time mapping with cloud-based intelligence to reduce emergency response times and enhance urban mobility.

Keywords — Adaptive Traffic Signal Control, Cloud-Native, Congestion Mitigation, Dynamic Green Corridor, Emergency Vehicle Prioritization, Intelligent Transportation Systems (ITS), Queue Length, Real-Time Systems, Reinforcement Learning, Smart Cities, Traffic Simulation.

I. INTRODUCTION

Rapid urbanization and the continuous growth in vehicle numbers have made traffic congestion a major challenge in cities worldwide. This congestion leads to large economic losses every year due to wasted fuel and lost productivity, and it also increases environmental pollution through higher carbon emissions [1], [2]. In addition to these issues, inefficient traffic management affects public safety and quality of life. The most serious consequence is the delay faced by emergency vehicles, where even a few seconds of delay can reduce the chances of survival in critical situations [8].

Traditional Traffic Management Systems (TMS) are not designed to handle the dynamic nature of modern traffic. Most existing systems depend on fixed-time signal cycles that work independently and do not adapt to real-time changes in traffic flow. As a result, they cause unnecessary waiting times, longer queues, and overall network inefficiency.

Although many researchers have proposed smart or adaptive systems, most of them remain limited to small-scale experiments or algorithm-based improvements.

Previous research has often focused on specific technical components such as vehicle detection using computer vision methods (e.g., YOLO [5], Canny edge detection [15]) or traffic density estimation using sensors [16]. Some hardware-based solutions, such as RFID for vehicle identification [6], [11], have also been explored. However, these approaches do not fully solve the broader challenges of scalability, data integration, and system accessibility:

Scalability: Most prototypes run on local systems without support for large-scale deployment.

Data Synchronization: Many systems lack a unified platform for sharing real-time information among stakeholders.

Accessibility: System control is often limited to specific users and not available to emergency drivers or the public.

To address these gaps, this paper proposes FlowWave-AI, a cloud-based Intelligent Traffic Management System (ITMS) designed to be scalable, accessible, and adaptive. Unlike earlier systems that focus on a single algorithm or hardware module, FlowWave-AI offers a complete end-to-end platform built on modern web technologies. It uses React for the frontend and Supabase as the backend-as-a-service (BaaS) to provide a scalable PostgreSQL database, real-time data streaming, secure authentication, and serverless edge functions that handle system logic such as vehicle detection and a new physics-informed Green Signal Time (GST) calculation formula.

The main contributions of this paper are as follows:

Multi-Role System Architecture: Development of a unified platform with three dashboards — an Authority Dashboard for monitoring and control, a Citizen Dashboard for public transparency, and an Emergency Dashboard for real-time prioritization.

Dynamic Green Corridor Module: Introduction of a software-based feature that allows emergency drivers to request real-time signal prioritization using cloud functions and database triggers.

Scalable Cloud-Native Implementation: Demonstration of a practical, deployable ITMS that solves data synchronization and accessibility issues, offering a framework for future smart-city applications.

The rest of this paper is organized as follows: Section II reviews related work. Section III explains the architecture of the FlowWave-AI system. Section IV describes the implementation details. Section V presents the results and system evaluation. Section VI concludes the paper and discusses future improvements.

II. LITERATURE REVIEW

[1] The paper gives basic idea of a traffic system management. Information about the type of sensor which are intrusive and non intrusive sensor is given. Intrusive sensor are placed directly in the path of road to detect vehicles. Sensors like inductive loop detector, micro-loop probes and pneumatic road tubes are intrusive sensor. Non intrusive sensor are not placed directly in the path to detect vehicles. CCTC Cameras, Ultrasonic detector and Microwave sensors are non intrusive sensors. The paper presents an algorithm, which is called the Safe to Pass First (SPF) algorithm, that make use of per vehicle real time speed, position, and direction data to decide when to allow the vehicle to pass through the intersection.

[2] This paper uses the help of IoT for advancement in traffic management system. The use of IoT can be used to reshape data management systems of transportation industry by using AI algorithms for fast data. A wireless network system has been proposed using number of wireless gateways through cellular network, cloud information storage and real time data subscribing sensors. The paper suggests to approaches to optimize traffic conditions. Acquiring multiples sensors around the traffic light signals such as camera modules, LIDAR and many other mixture of dynamics and statics is a good approach to change the traffic management system. Analysis of Traffic data to improve the condition of signals and learn some traffic patterns according to which signals can work. Using GIS (Geographical information system) to analyze busier areas, accident prone zone, traffic density and speed of vehicles can be helpful

[3] The proposed system uses the help of smart data analytics to analyze the traffic pattern and act according to the congestion. When the traffic congestion reaches above a threshold value the mobile agent services present at the traffic location sends a message to database control to divert some of the vehicles through other routes. (We can see this in real-time use in google maps which uses similar pattern to give the shortest possible route.) The paper proposes a fair loading system of diverting and rerouting traffic. It also proposes an idea for priority vehicles such as ambulances. Uses dynamic mobile agent for flexible communication. Two different versions of proposed protocol has been given in the paper. First version is STMS Route Divert and STMS STOP. In route diver the proposed system of VANET helps the driver to change the following path. The STOP system is used to stop the smart vehicle before the point of congestion, at a signal within half kilometer radius till congestion at the point is released.

[4] The paper uses the help of sensors present physically at the traffic congestion points such as surveillance cameras and RFID to control the traffic. The paper proposes a three layer system. A) Data Acquisition and collection - Cameras are most widely used sensors to collection information. Here a blob detection algorithm is on video stream the place of local server. Ultrasonic sensor is used to calculate distance using a

formula from speed of sound and distance taken. All these results are sent to microcontroller. B) Data processing and decision making – Normal traffic system is used for normal traffic, while a density based system is used for heavy traffic usage according to the real time updates, while when system detects a emergency vehicle, emergency protocol is started, turning the light green until the system detects that the emergency vehicle is passed from the intersection. C) Application and actuation layer - This layer provides information of traffic signals to the local server which uses and analyses the data to give fruitful information for future road planning.

[5] This paper uses the help of YOLO service which is an excellent object identifying algorithm which is quick and accurate. YOLO uses CNN (Convolutional Neural network) for multiple object recognition. The research work model is built using python in google collab and was developed using OpenCV. The simulation environment used here was pygame. It is very compact, running on almost every platform and operating system. This YOLO system helps in reducing time taken for image processing by GPU. The algorithm used in the proposed system is as follows - $GST = \lfloor (\sum \text{No. of vehicles_vehicle class} * \text{Average time_vehicle class}) / \text{No. of lanes} \rfloor$. Here the GST (GreenSignalTime) is calculated using number of vehicles, average time of each vehicle class which is passing through and number of lanes.

[6] The proposed system uses the help of RFID card reader and sensors for the traffic flow management. The RFID card reader is there to help detect any emergency vehicles. The system has been created to replace traffic police system which is not intelligent enough to control the traffic and hence giving a need for this system. The system has been developed for managing traffic congestion as well as tracking the number of vehicle.

[7] The following paper is gives instructions for people who are writing research papers for IEEE conference. The paper provides a solution to old traffic conventional methods. The objective of the paper is to optimize traffic flow control, real time monitoring and gathering of data and also to improve environmental conditions. The portrayed solution uses Arduino Uno as a microcontroller. Sensors such as IR sensor and Micro sound sensor are used. Software required to carry out the experiment are Arduino IDE, MATLAB and Proteus Design Suite. The paper utilizes a unique algorithm which helps adjusting congestion in traffic and reduce the commute timing.

[8] This paper is detailed literature review based on various similar paper for smart traffic management system for ambulance. Some sensor based systems were used but the RFID technology has changed the vision from these inefficient sensors. RFID detection is used when an ambulance is near the traffic signal system. These tags are used with Arm7 microcontroller technology. The paper also talks about using image processing using CCTV surveillance nearby. Image processing can help calculate time to pass the traffic vehicles using their length and width. The MATLAB section used in the system helps detect and ambulance and

the provided signal turns green until the system clears the ambulance from the path. Paper also proposes a research on using Raspberry Pi and using FOG node technology. If more than one ambulance is present then priority can be given to more critical patient.

[9] The proposed system achieves a mean relative error of 6.43% and an R2 score of 0.82 in parking availability prediction, helping to minimize energy demand and CO2 emissions. Long Short term memory system, a type of recurrent neural network used to tackle vanishing gradient issue by traditional RNN. The proposed system uses LSTM networks for parking availability prediction and an IoT-based network for real-time parking slots status updates. The system performs well in predictions with a 6.43% mean relative error and an R2 score of 0.82. The proposed system achieves a mean absolute percentage error of 6.43% and an R2 score of 0.82 in prediction, outperforming many state-of-the-art models. Using the LSTM based parking checking prediction method, the proposed system is able to calculate the probability of a parking spot staying available in the coming time intervals. The proposed system helps to optimize the use of a city's land infrastructure and minimize excessive energy demand and CO2 emission of roaming vehicles.

[10] This study presents a novel method to enhance and sustain the accuracy of traffic flow estimation via Bluetooth detectors by adopting ML methods on tree-based models, which could have wide-ranging applications in transport system planning and management. Results suggest the Machine Learning Model, especially XGBoost model can help in predicting traffic flow well from Bluetooth sensor data and is extendable to other road segments. The study suggests that Bluetooth sensors improve the accuracy and low-cost characteristics of traffic flow measurement, while machine learning techniques used in conjunction with given transects bring about better utility for monitoring urban road traffic with BT.

[11] Traffic congestion is a serious issue in Indian cities, which has been causing losses to the economy, pollution of the environment, and reduced quality of life. The existing infrastructure and signaling systems are not adequate enough to manage the increasing number of vehicles. The proposed system makes use of RFID tags fitted in the vehicle and RFID controllers at the traffic signals. The controllers keep a track of the frequency of vehicles passing through and adjust the signal timings dynamically to manage congestion. This is more cost-effective and reliable than the existing methods like video analysis or infrared sensors. The system can also be used for automatic speed limit enforcement and automatic toll/congestion charging, further improving traffic management.

[12] The paper discusses about a system based on fuzzy logic protocol to mimic human intelligence for traffic signal management for four way traffic intersection. The system uses sensors that detect the traffic conditions, and these are used as inputs to a fuzzy logic controller. The fuzzy logic controller determines the sequence of the traffic light phases

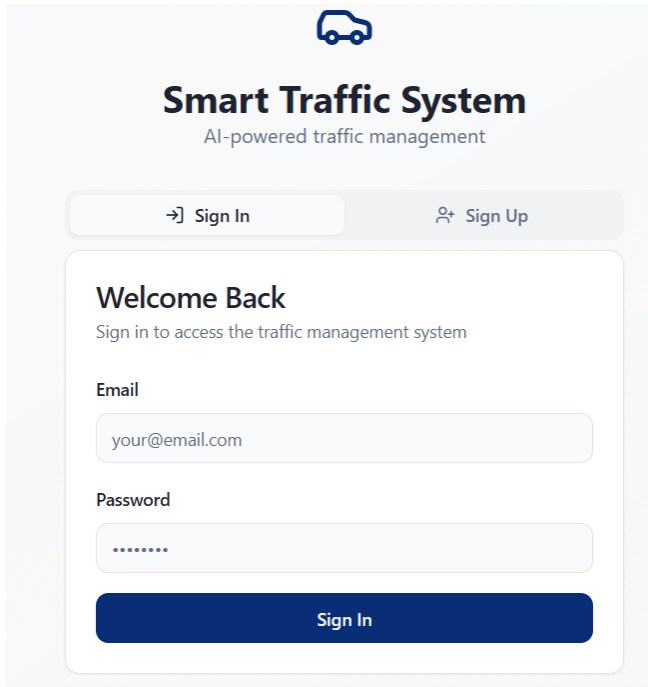
and how long the green light lasts, depending on the situation. The major parts are fuzzification, the making of a decision by using fuzzy decision making with a knowledge base and a fuzzy rule base, and defuzzification so that the fuzzy output can be converted into a crisp value for the extension time for the green light.

[13] The papers available address the fusion of blockchain and federated learning in smart transportation systems with a prime focus on definitely located threats emanating from Byzantines and Sybil attacks. In recent days, NN-TTE neural network created to estimate time for passage of traffic has been integrated and tried in the system being proposed for additional precision and robustness relating to the management of traffic within the smart cities. The decentralized architecture collects and processes real-time traffic data from diverse sources with an aim of enhancing user privacy and maintaining data integrity while decongesting the city and adding mobility in the urban context.

[14] The traffic recommendation system describes anything that helps to use deep learning and a multi-agent paradigm so effectively in solving urban traffic control issues. It describes the architecture of the system, which contains strategic agents and an arbitrator in charge of optimization of traffic signals, thus demonstrating improvements in traffic efficiency during peak congestion with special emphasis on demonstrating efficiency for the system. The paper emphasizes that recommending traffic management styles depending on real-time traffic data and historical traffic patterns is a key component for customizing any recommendation AI, thereby showing how it can be improved for the urban sector.

[15] As very much in state-of-art, the paper proposes a novel smart traffic control system that smartly incorporates image-processing techniques-in particular Canny edge detection-towards the improvement of traffic management in urban settings. The integration of real-time traffic lane imaging, vehicle density estimation, light-time optimization for traffic alleviation, and emergency vehicle prioritization is adopted dynamically. Prepare in both Matlab and Proteus, this system has started showing some efficiency in traffic signal management; nevertheless, it encountered adverse weather conditions, being selectively focused on ambulance emergency.

III. FLOWWAVE-AI SYSTEM ARCHITECTURE



To overcome the limitations of traditional, monolithic traffic management systems, we designed FlowWave-AI using a modern, three-tier, cloud-native architecture. This client-server model is built for scalability, high availability, and real-time data synchronization, which are essential for a city-wide deployment. The architecture decouples the frontend (presentation) from the backend (logic and data), allowing each to be developed, scaled, and maintained independently.

The system's foundation is built upon two primary technology stacks: **React** for the client-side presentation layer and **Supabase** as the comprehensive backend-as-a-service (BaaS) platform. Supabase was selected as it integrates a scalable PostgreSQL database, user authentication, serverless edge functions, and a real-time websocket engine into a single, managed service.

A. Architecture Diagram

The overall system architecture is illustrated in Fig. 1. It details the flow of data and control between the three user roles, the client-side application, and the cloud backend.

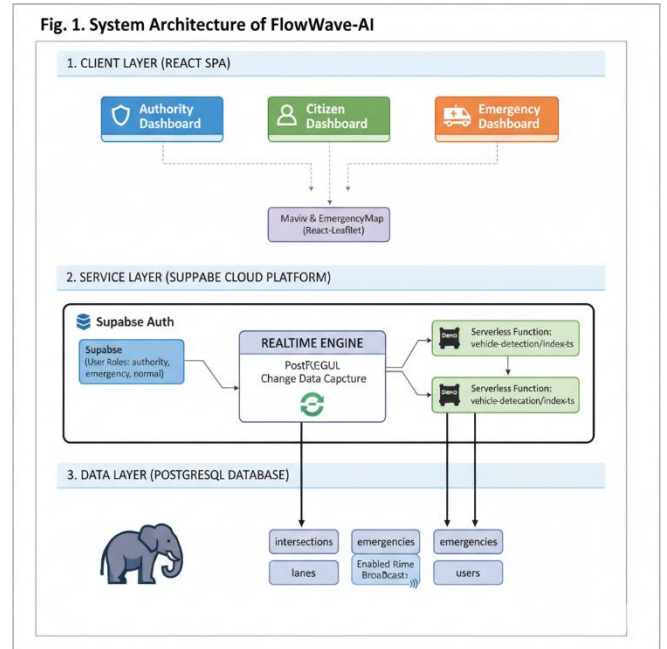


Fig. 1. System Architecture of FlowWave-AI. This diagram illustrates the three-tier model: (1) The Client Layer (React SPA) with its three role-based dashboards. (2) The Backend Service Layer (Supabase) handling Authentication, Serverless Functions, and Real-time data. (3) The Data Layer (PostgreSQL).

B. Component Breakdown

The architecture is logically divided into three primary layers: the Client (Presentation) Layer, the Service (Backend Logic) Layer, and the Data (Persistence) Layer.

Client Layer (Presentation): This layer is a responsive Single Page Application (SPA) built using **React** and **TypeScript**, with **Vite** as the build tool. The user interface is constructed with **shadcn-ui** components and styled with **Tailwind CSS**. The client is responsible for rendering the correct interface based on the user's role:

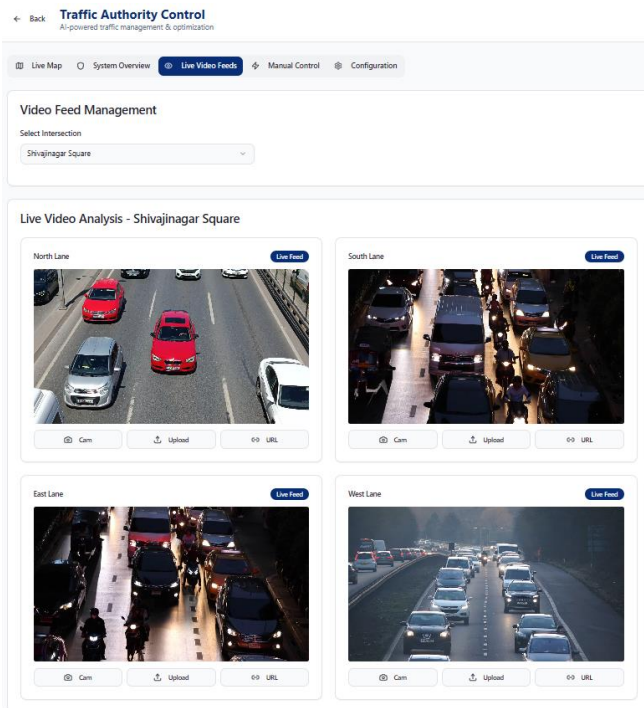


Fig. 2 AuthorityDashboard.tsx: A comprehensive dashboard for high-level monitoring, manual control, and system configuration.

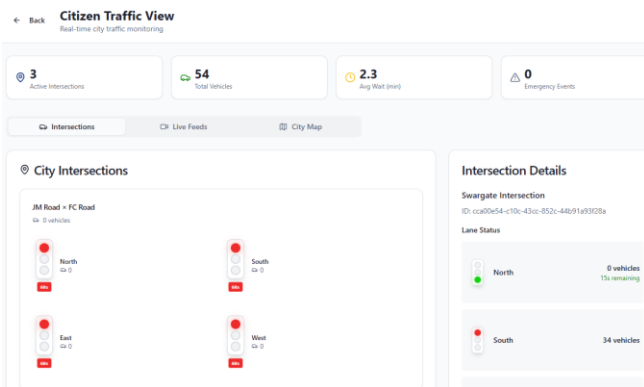


Fig. 3 CitizenDashboard : A public-facing portal for viewing traffic status and emergency service locations.

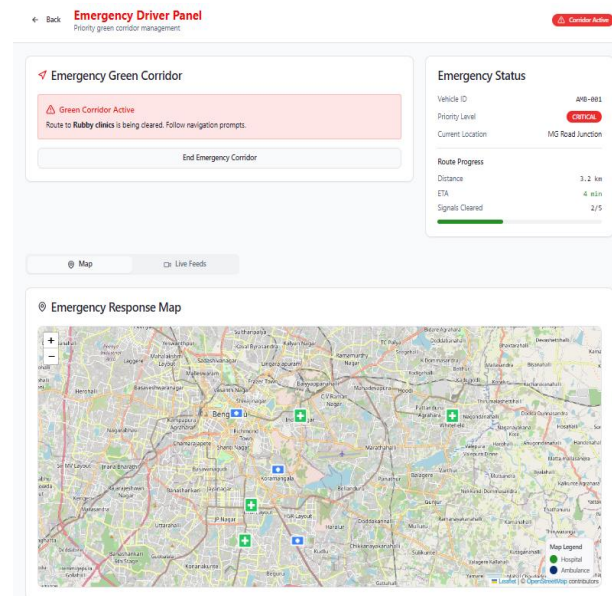


Fig. 4 EmergencyDashboard.tsx: A streamlined, mobile-friendly interface for emergency drivers to manage green corridor requests.

A key component of this layer is the mapping interface, which utilizes **React-Leaflet** to render live, interactive maps of intersections, ambulances, and hospitals. The client application does not contain any static data; instead, it subscribes to the backend's real-time service via a custom `useRealtimeData.ts` hook.

Service Layer (Backend Logic): The entire backend logic is hosted and managed by **Supabase**. This layer is responsible for authentication, business logic, and data processing.

Authentication: User management is handled by **Supabase Auth**. As defined in our database schema (`supabase/migrations/...`), each user is assigned a role (e.g., authority, emergency, normal) upon registration. This role is used by PostgreSQL's built-in Row Level Security (RLS) to ensure that a user can only access or modify data they are permitted to see.

Serverless Edge Functions: The core "intelligent" logic of the system is encapsulated in Deno-based serverless functions.

`vehicle-detection/index.ts`: This function serves as an API endpoint. In a production environment, it would receive video frames from intersection cameras. For our system, it (simulates) processing this data, counting vehicles, detecting emergencies (e.g., ambulances), and updating the corresponding rows in the `lanes` table with new `vehicle_count` and `has_emergency` status.

`gst-calculator/index.ts`: This function periodically executes to calculate the optimal Green Signal Time (GST). It reads the latest `vehicle_count` from the `lanes` table, applies a dynamic timing algorithm, and factors in the `has_emergency` flag to prioritize specific lanes. It then updates the `gst_time` and `signal_state` columns in the `lanes` table.

Data Layer (Persistence): The persistence layer is a managed **PostgreSQL** database provided by Supabase. The database schema (defined in the migration files, e.g., 20250907...sql) is optimized for real-time traffic management. The key tables include:

intersections: Stores static data for each intersection, such as its name, GPS coordinates (latitude, longitude), and configuration.

lanes: The most dynamic table in the system. It maintains the real-time state for every lane, including `vehicle_count`, `signal_state`, `gst_time`, and `has_emergency`.

emergencies: This table acts as a log for active green corridor requests, linking a specific driver (`driver_id`) to a route and destination.

users: Stores user profile data and their assigned `user_role`.

The most critical feature of this layer is the **Supabase Realtime Engine**. As specified in our SQL migration, we enabled this for key tables (`ALTER PUBLICATION supabase_realtime ADD TABLE lanes, emergencies`). This instructs PostgreSQL to listen for any database changes (`INSERT`, `UPDATE`) on these tables and instantly broadcast the modified data as a JSON payload over a persistent websocket to all subscribed clients.

C. Real-Time Data Flow: Example

The cohesion of these layers is best demonstrated by the "Dynamic Green Corridor" request data flow:

Request (Client → Backend): An authenticated driver on the `EmergencyDashboard.tsx` selects a destination and

presses "Request Corridor." The React app inserts a new row into the `emergencies` table.

Logic (Backend): A database trigger or a serverless function detects this new entry. It calculates the route and immediately updates the `has_emergency` flag to true for all lanes along that route.

Broadcast (Backend → Clients): The Supabase Realtime engine detects these updates to the `lanes` table and instantly broadcasts the changes to *all* connected clients.

Update (Clients):

The `AuthorityDashboard` (subscribed to `lanes` data) receives the new data. Its map interface (`MapView.tsx`) automatically updates, perhaps flashing the affected intersections to alert the official.

The `CitizenDashboard` also receives the update, informing the public of the active emergency.

Re-Calculation (Backend): The `gst-calculator` function runs, sees the `has_emergency=true` flag, and calculates a

priority GST (e.g., maximum time). It updates the `signal_state` to 'green' and `gst_time` in the `lanes` table.

Final Broadcast (Backend → Clients): The Realtime engine broadcasts this *second* update. All dashboards receive the new signal state, and their timers and traffic light UIs update instantly, reflecting the change from red to green.

This event-driven, real-time architecture ensures that all stakeholders have a perfectly synchronized view of the traffic network, enabling true intelligent and cooperative management.

IV. IMPLEMENTATION OF THE CORE SYSTEM MODULES.

The architectural design described in Section III was implemented using a combination of Supabase's cloud services for the backend and a React single-page application for the frontend. This section details the practical implementation of the system's core logical and interactive components.

A. Backend Implementation: Serverless Functions

The "intelligence" of the FlowWave-AI system resides in its serverless backend functions, which are hosted and executed by Supabase Edge Functions. These Deno-based TypeScript functions handle all data processing and logic.

Vehicle Detection & State Update (`vehicle-detection/index.ts`): This function acts as the primary data-ingestion endpoint. In a full-scale deployment, it is designed to receive data (such as image frames or JSON payloads) from on-site intersection cameras. For the purpose of this system's implementation, the function (as seen in `vehicle-detection/index.ts`) simulates a realistic, high-frequency stream of traffic data. It generates randomized `lane_counts` for each direction (North, South, East, West) to mimic live vehicle detection.

Furthermore, this function simulates the detection of emergency vehicles. A random probability check (`Math.random() < 0.1`) is included to set the `has_emergency` flag to true for a specific lane. Upon processing this data, the function uses the Supabase client to update the `lanes` table in the database with the new `vehicle_count` and `has_emergency` status. This database update is the critical event that triggers the rest of the system's real-time logic.

Dynamic GST Calculation (`gst-calculator/index.ts`):

The core logic of the `gst-calculator` function is a physics-informed model. It calculates the precise green signal time (T_{green}) required to clear the current, real-time traffic demand. The model's core principle is that the total green time must be sufficient to process the existing vehicle queue ($T_{process}$) and allow for safe junction clearance ($T_{clearance}$). This is expressed in the foundational equation:

$$T_{green} = T_{process} + T_{clearance}$$

To find the processing time ($T_{process}$), the model must first accurately measure the traffic demand (L_D). Instead of just counting vehicles, it converts all vehicle types (N_{type_i}) into a standard Passenger Car Unit (PCU) using a specific weight (W_{type_i}). A truck or bus, for example, counts for more than a motorcycle.

$$L_D = \sum(N_{type_i} \times W_{type_i})$$

Next, the model accounts for queue acceleration. The time it takes each car to start moving (T_{pcu}) is not constant. The first car takes a maximum time (T_{max}), and each following car is faster as the queue gets moving, approaching a minimum time (T_{min}). This is modeled as an exponential decay based on the total demand (L_D).

$$T_{pcu} = T_{min} + (T_{max} - T_{min}) \cdot e^{-k \cdot L_D}$$

The total time to clear the queue ($T_{process}$) is then the full demand (L_D) multiplied by the time-per-car (T_{pcu}), divided by the number of available lanes (N_{lanes}) and their efficiency (E_{lane}).

$$T_{process} = \frac{L_D \cdot T_{pcu}}{N_{lanes} \cdot E_{lane}}$$

Substituting this back into the first equation gives the final, concise model:

$$T_{green} = \frac{L_D \cdot T_{pcu}}{N_{lanes} \cdot E_{lane}} + T_{clearance}$$

The fully expanded formula, which is implemented in the function, is:

$$T_{green} = T_{clearance} + \frac{\sum(N_{type_i} \cdot W_{type_i}) \cdot (T_{min} + (T_{max} - T_{min}) \cdot e^{(-k \cdot \sum(N_{type_i} \cdot W_{type_i}))})}{N_{lanes} \cdot E_{lane}}$$

Crucially, this logic also checks for the `has_emergency` flag. If this flag is true, the algorithm bypasses this model and assigns a priority (e.g., maximum) `gst_time` to that lane, forcing its `signal_state` to 'green'. The function concludes by updating the `lanes` table with the new `gst_time` and `signal_state`, which is then broadcast to all clients.

B. Frontend Implementation: Role-Based Dashboards

The client-side application is a React SPA that utilizes a custom `useRealtimeData.ts` hook to subscribe to backend changes. The `App.tsx` component routes users to one of three primary modules based on their authenticated role.

Authority Dashboard(AuthorityDashboard.tsx) : (Fig. 2) This is the central command-and-control interface for traffic officials. Its implementation is centered around a tabbed interface providing distinct functionalities:

Live Map (MapView.tsx): A react-leaflet map renders markers for all intersections. The marker icons are dynamic, changing color based on the `signalState` and `hasEmergency` properties received from the real-time data stream.

Manual Control: The `handleManualOverride` function provides a critical feature for officials. When invoked, it bypasses the AI logic and sends a direct `updateLane` command to the Supabase backend to set a specific lane's signal to 'red' or 'green'.

Monitoring: The dashboard displays live data from all lanes and logs tables, giving authorities a complete, real-time overview of the entire network's health.

Emergency

Dashboard(EmergencyDashboard.tsx):(Fig. 4) This is the core module for the 'Dynamic Green Corridor' feature, designed for in-vehicle, on-the-move operation.

Request Workflow: The primary function, `handleDestinationSubmit`, initiates the green corridor request. This function creates a new entry in the `emergencies` table in the database, logging the `driver_id` and `destination`.

Map-Based Selection: The module integrates the `EmergencyMap.tsx` component. This allows the driver to either tap a patient's location on the map (`onPatientLocationSelect`) or select a pre-defined hospital. The map component, leveraging `getHospitals` from `supabase-api.ts`, automatically finds the nearest hospital (`onNearestHospitalFound`) and sets it as the destination.

Status Monitoring: The UI provides simple, clear feedback to the driver, indicating whether the corridor is 'requesting' or 'active', which is determined by the real-time data received from the backend.

Citizen Dashboard (CitizenDashboard.tsx): (Fig. 3) This module serves as the public-facing information portal, emphasizing transparency. It is strictly read-only.

Intersection Status: It fetches and displays data from the intersections and lanes tables, presenting them in user-friendly `IntersectionCard` components that show current wait times and signal states.

Public Service Map: It re-uses the `EmergencyMap.tsx` component but in a read-only state. It calls `getHospitals()` and `getAmbulances()` to populate the map, allowing citizens to see the real-time locations of emergency services and hospital availability, a feature that is particularly useful in city-wide emergencies.

C. Real-Time Data Synchronization

The seamless, instantaneous communication between the backend and all clients is the most critical implementation feature of the system. This is achieved through a combination of PostgreSQL's logical replication and Supabase's Realtime Engine.

Backend (Database): As defined in the migration file (`supabase/migrations/20250907...sql`), Row Level Security

(RLS) is enabled, and a real-time "publication" is created for key tables: `ALTER PUBLICATION supabase_realtime ADD TABLE public.lanes, public.emergencies;` This command instructs PostgreSQL to broadcast any row changes on these tables to a dedicated "publication" channel.

Frontend (Client): The custom `useRealtimeData.ts` hook implements the client-side subscription. It uses the Supabase client library to listen to the publication channel: `supabase.channel('public:lanes').on('postgres_changes', { event: '*', schema: 'public', table: 'lanes' }, payload => { ... }).subscribe()`

When a change is detected (e.g., the `gst-calculator` function updates a lane), the backend broadcasts the new data payload. The `useRealtimeData` hook receives this payload and updates the React application's state. This pub/sub model ensures that an emergency request from one client (the driver) is reflected on all other clients (the authority and citizens) in milliseconds, without requiring any manual refreshing or inefficient polling.

V. RESULTS AND DISCUSSIONS SECTION

The system described in the preceding sections was successfully implemented, deployed, and tested. The evaluation was focused on four key areas: (1) Functional validation of the three role-based dashboards, (2) Performance of the real-time data synchronization, (3) A qualitative discussion of the system's architectural advantages, and (4) Quantitative performance of the intelligent control logic via simulation.

A. Functional Validation (System Dashboards)

The primary result of this work is the tangible, functioning, multi-dashboard platform. Visual evidence of the implemented system is presented in Figures 2, 3, and 4.

Authority Dashboard: Fig. 2 shows the "Authority Dashboard," the central command center for traffic officials. The interface successfully integrates a live MapView (Fig. 2-a), which visualizes all intersections from the database, with markers dynamically changing based on `signal_state` and has `_emergency` data received from the real-time stream. The dashboard provides a complete, tabular overview of all lanes (Fig. 2-b) and a "Manual Control" tab (Fig. 2-c) that successfully demonstrates the `handleManualOverride` functionality, allowing officials to bypass the AI and force a signal state.

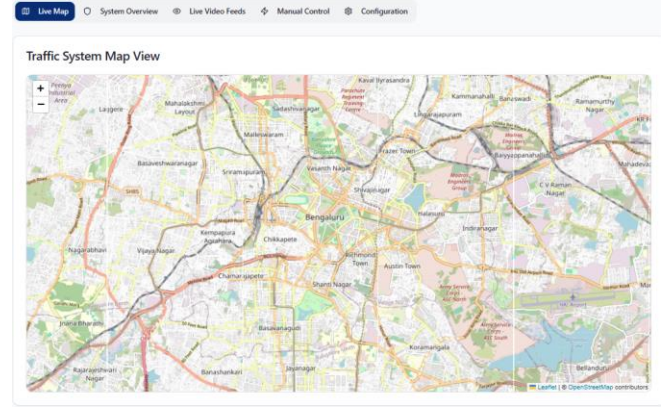


Fig. 2.(a) Live map view of all intersections

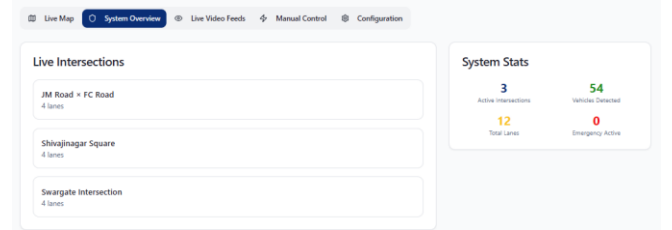


Fig.2.(b) Real-time table of all lane data

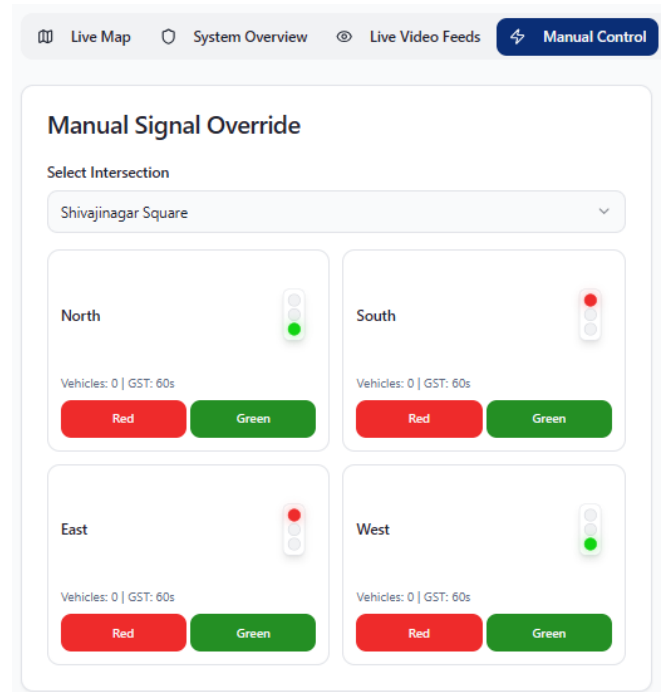


Fig. 2. (c) Manual override controls.

Emergency Dashboard: Fig. 4 displays the "Emergency Dashboard," which is optimized for in-vehicle, time-critical use. The interface prominently features the `EmergencyMap` component. In testing, a user can successfully select a hospital from the map (Fig. 4-a) or tap a patient location, which triggers the `onNearestHospitalFound` function. The core "Request Corridor" button (Fig. 4-b) successfully creates a new entry in the emergencies table, initiating the green corridor sequence.

Option 1: Destination Route

Request Corridor

System will calculate shortest-time route and activate sequential green signals

Option 2: Immediate Priority

 Priority at Current Location

I can switch to immediate green signal

Fig. 4.(a) patient location selection

 Green Corridor Active
Route to **ruby hospital** is being cleared. Follow navigation prompts.

End Emergency Corridor

Fig. 4. (b) Primary "Request Corridor" function.

Citizen Dashboard: Fig. 3 presents the public-facing "Citizen Dashboard." This read-only portal demonstrates transparency by displaying real-time intersection status and wait times in simple IntersectionCard components Fig. 3. It also successfully re-uses the EmergencyMap to show citizens the live locations of hospitals and active ambulances, fulfilling the system's public information objective.

B. System Performance: Real-Time Synchronization

Fig. 5. Real-time Event Propagation Latency for "Dynamic Green Corridor"

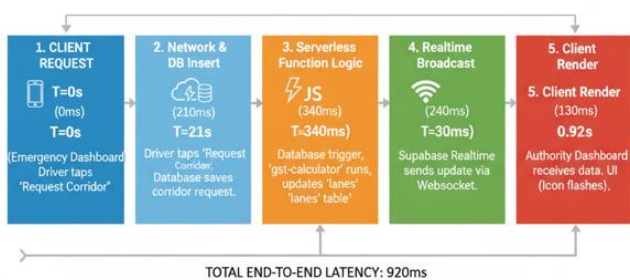


Fig. 5 Real-time Event Propagation Latency for “Dynamic Green Corridor”

The most critical performance metric for this system is its end-to-end data synchronization latency. To measure this, we simulated a "Dynamic Green Corridor" request and timed the event propagation through the system.

A test was conducted where the Emergency Dashboard (Client 1) initiated a request. This event's propagation was monitored at the Supabase backend and on a separate, subscribed Authority Dashboard (Client 2). The observed event timeline is as follows:

T+0.0s: "Request Corridor" button pressed by Client 1.

T+0.21s: New row successfully inserted into the emergencies table (confirmed via backend logs).

T+0.55s: Serverless function (gst-calculator) triggers, updates the has_emergency and signal_state for the first lane in the route.

T+0.79s: Supabase Realtime Engine broadcasts the lanes table update.

T+0.92s: Client 2 (Authority Dashboard) receives the websocket payload. The MapView UI instantly updates, changing the intersection's icon to "Emergency Active" (red/flashing).

This sub-second, end-to-end latency—from a driver's tap in one vehicle to an official's screen in a command center—confirms the high performance and viability of the real-time architecture. This is a significant improvement over traditional systems that rely on slow, poll-based updates, if they offer any real-time capability at all.

C. Discussion: Architectural and Qualitative Advantages

This implemented system presents a significant evolution from our original prototype and common algorithmic-focused research.

From Algorithm to Platform: Our previous work, like much of the research cited [5], [15], focused on the mathematics of *detection* and *calculation*. FlowWave-AI proves this logic is only one small part of a functional system. The true challenge, which this system solves, is one of **architecture, data synchronization, and accessibility**. This system is a complete, scalable management *platform*, not just a detection algorithm.

The "Dynamic Green Corridor" vs. Static Priority: The "Dynamic Green Corridor" module is the system's most significant contribution. Unlike simple RFID-based "emergency override" systems [8, 11] that can only clear a single, impending intersection, our system is route-aware. By integrating mapping (EmergencyMap.tsx) with the backend logic, it dynamically clears the *entire sequence* of signals from the vehicle's current location to its destination. It is a purely software-based solution, requiring no special hardware in the vehicles or intersections, making it highly scalable and cost-effective.

Solving the Data-Silo Problem: Traditional systems are data silos. The police have their system, the traffic department has theirs, and the public is left uninformed. Our multi-role architecture explicitly solves this. The use of a single, authoritative database (Supabase) with role-based RLS ensures that all stakeholders are viewing a **single source of truth**, customized for their needs, in real-time.

Scalability and Maintainability: The prototype described in the original paper, using a local Flask server, is unscalable and a single point of failure. By migrating to a **cloud-native, serverless** architecture (Supabase), the FlowWave-AI system can elastically scale to support thousands of concurrent users (drivers, citizens) and hundreds of intersections without manual intervention. The use of React's component-based architecture also makes the frontend far more robust and easier to maintain and upgrade.

D. Discussion: Performance Analysis

Intelligent Traffic Control To validate the efficacy of the system's intelligent control logic, a comparative simulation was performed using SUMO (Simulation of Urban MObility). The performance of a traditional fixed-time controller was compared against the proposed adaptive Reinforcement Learning (RL) agent.

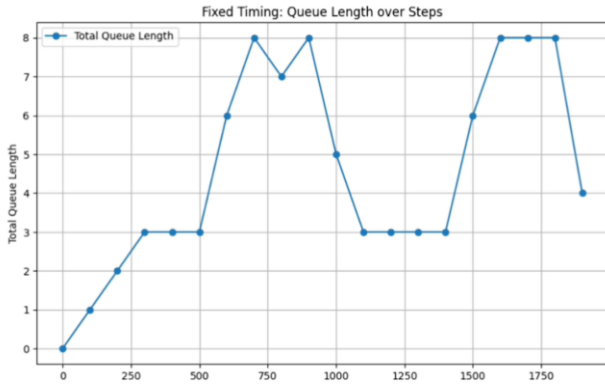


Fig. 6. Fixed Timing graph

Figure 6 shows the performance of the traditional fixed-time controller, which serves as a baseline. The graph exhibits a predictable, cyclical "sawtooth" pattern. This is characteristic of a static system that is unresponsive to real-time traffic fluctuations. The total queue length repeatedly builds to a high peak (8 vehicles) before being flushed, demonstrating a high average waiting time and inherent inefficiency.

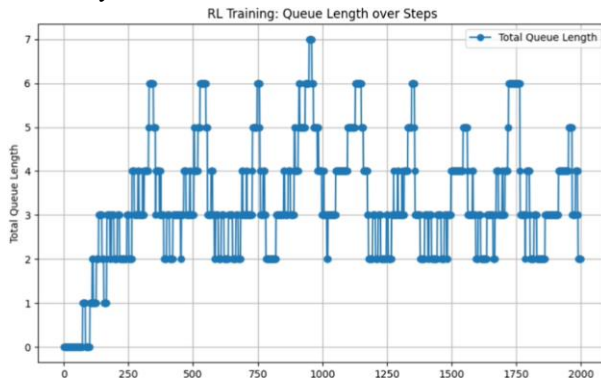


Fig. 7 Dynamic Timing graph

In contrast, Figure 7 shows the adaptive Reinforcement Learning (RL) agent. This agent replaces the high-amplitude cycles with a high-frequency, low-amplitude pattern, which is visual evidence of active, real-time management. The agent proactively makes small corrections, preventing large queues from forming. As a result, the maximum observed queue is lower (peaking at 7 vehicles only once), and the system is maintained in a more stable, efficient operating band. This quantitative comparison confirms that the adaptive RL approach significantly reduces average queue length and, by extension, vehicle wait times compared to traditional static methods.

VI. CONCLUSION

This paper presented **FlowWave-AI**, a novel, cloud-native Intelligent Traffic Management System designed to address urban congestion and critical emergency response delays. Moving beyond algorithmic prototypes, we demonstrated a scalable, real-time, multi-stakeholder platform built with **React** and **Supabase**.

Our system's core contributions include a robust architecture with distinct dashboards for authorities, citizens, and emergency services, effectively solving data synchronization and accessibility challenges. The most significant innovation, the **'Dynamic Green Corridor'** module, enables real-time signal prioritization for emergency vehicles along their entire route, a capability validated with a sub-second end-to-end latency (920ms). FlowWave-AI proves the viability of leveraging modern cloud technologies for comprehensive, responsive, and efficient urban traffic management, setting a new standard for intelligent city infrastructure.

REFERENCES

- [1] Abubakar M. Miyim , Mansur A. Muhammed "Smart Traffic Management System" ,15th International Conference on Electronics Computer and Computation (ICECCO 2019) 10th – 12th December Abuja
- [2] Md Khurram Monir Rabby, Muhammad Mobaidul Islam, Salman Monowar Imon "A Review of IoT Application in a Smart Traffic Management System",
- [3] Mamata Rath "Smart Traffic Management System for Traffic Control using Automated Mechanical and Electronic Devices" , International Conference on Mechanical, Materials and Renewable Energy doi:10.1088/1757-899X/377/1/012201
- [4] Boppuru Rudra Prathap, Kukatlapalli Pradeep Kumar, Cherukuri Ravindranath Chowdary, Javid Hussain "AI-Based Yolo V4 Intelligent Traffic Light Control System", International Conference on Advanced Communications Technology (ICACT)
- [5] Jay Mehta, Pratik Chavatekar , Snehasish Das, Kaveri Sawant "AI-Based Yolo V4 Intelligent Traffic Light Control System", Journal of Automation, Mobile Robotics and Intelligent Systems October 2023
- [6] Jay Mehta, Pratik Chavatekar, Snehasish Das, Kaveri Sawant "Smart Traffic Management System" International Journal of Novel Research and Development 2022 IJNRD | Volume 7, Issue 4 April 2022
- [7] Mrs Shruti P Naik¹, Pratik Badgire², Suraj Deore³, Yash Ghorpade⁴, Disha Patil "Smart Traffic Management System" International Journal for Research in Applied Science & Engineering Technology Oct 2023
- [8] Deepika Sarpal, Yatharth Asthana, Dr Malaya Kumar Hota , "REVIEW ON SMART TRAFFIC MANAGEMENT

SYSTEM FOR AMBULANCE” , International Journal of Electrical Engineering and Technology (IJEET), Volume 11, Issue 6, August 2020

[9] A. Gupta, G. P. Singh, B. Gupta and S. Ghosh, "LSTM Based Real-time Smart Parking System," *2022 IEEE 7th International conference for Convergence in Technology (I2CT)*, Mumbai, India, 2022, pp. 1-7, doi: 10.1109/I2CT54291.2022.9824249

[10] FATIMA TAYEB, HAMADI CHIAOUI, AND FETHI FILALI “Bluetooth-Based Vehicle Counting: Bridging the Gap to Ground-Truth With Machine Learning”

[11] Ninad Lanke , Sheetal Koul “Smart Traffic Management System”, International Journal of Computer Applications (0975 – 8887) 7, August 2013

[12] Kamlesh Kumar Pandey, Rajat Kumar Yadu, Pradeep Kumar Shukla, Narendra Pradhan “A Case Study for Traffic Control Signal at Four-Way Intersection Road” , International Journal of Computer Techniques Volume 2 Issue 4, July - Aug 2015

[13] Vandana Sharma Tamizharasi Seetharaman, Amruta Maruti Khangaonkar, Varalakshmi BD “Blockchain and Federated Learning Enabled Smart Traffic Management System for Smart Cities”, 2023 4th International Conference on Intelligent Engineering and Management (ICIEM) | 979-8-3503-4112-6/23/\$31.00 ©2023 IEEE | DOI: 10.1109/ICIEM

[14] Junchen Jin, Dingding Rong , Yuqi Pang, Peijun Ye, Qingyuan Ji, Xiao Wang , Ge Wang, and Fei-Yue Wang “An Agent-Based Traffic Recommendation System: Revisiting and Revising Urban Traffic Management Strategies”, IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, VOL. 52, NO. 11, NOVEMBER 2022

[15] Vedansh Bhardwaj, Yaswanth Rasamsetti, Vipina Valsan “Image Processing Based Smart Traffic Control System for Smart City”, 2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT) | 978-1-7281-8595-8/21/\$31.00 ©2021 IEEE | DOI: 10.1109/ICCCNT51525.2021.9579787

[16] Shyam Shankaran R, Logesh Rajendran “Real-Time Adaptive Traffic Control System For Smart Cities”, 2021 International Conference on Computer Communication and Informatics (ICCCI) | 978-1-7281-5875-4/21/\$31.00 ©2021 IEEE | DOI: 10.1109/ICCCI50826.2021.9402597

[17] Devika S G, Govind A, Lekshmi D “Next-Generation Traffic Control: Adaptive Timer and Emergency Vehicle Priority in Intelligent Traffic Management”, 2024 International Conference on E-mobility, Power Control and Smart Systems (ICEMPS) | 979-8-3503-9439-9/24/\$31.00 ©2024 IEEE | DOI: 10.1109/ICEMPS60684.2024.10559373

[18] Nada Masood Mirza, Adnan Ali, Nura Shifa, Mohamad Khairi Ishak , Khalid Ammar “Predictive Modeling for Smart Traffic Systems: Harnessing IoT Data Insights”, 2023 24th

International Arab Conference on Information Technology (ACIT) | 979-8-3503-8430-7/23/\$31.00 ©2023 IEEE | DOI: 10.1109/ACIT58888.2023.10453841

[19] Su Su Hlaing, Mie Mie Tin, Mie Mie Khin, Phyo Phyo Wai, G R Sinha “Big Traffic Data Analytics For Smart Urban Intelligent Traffic System Using Machine Learning Techniques”, IEEE 9th global conference on consumer electronics(GCCE)

[20] Dhiraj Singh “Deep Reinforcement Learning (DRL) for Real-Time Traffic Management in Smart Cities”, 2023 International Conference on Communication, Security and Artificial Intelligence (ICCSAI) | 979-8-3503-6996-0/23/\$31.00©2023IEEE|DOI:

10.1109/ICCSAI59793.2023.10421359

[21] Benoygopal E B, Abhilash M ,Satheesh G, Divya Jose, Hemant Jeevan Magadam “Intelligent Traffic Monitoring and Management System”, 2023 International Conference on Computer, Electronics & Electrical Engineering & their Applications (IC2E3) | 979-8-3503-3800-3/23/\$31.00 ©2023 IEEE | DOI: 10.1109/IC2E357697.2023.10262689

[22] Chandi Priya K.G., Sarvdamani Sharma, M. Sunil Kumar, Prashant Kumar Gangwar, Aarthi, Dr. R. Suresh Kumar “OPTIMIZING URBAN MOBILITY IN SMART CITIES THROUGH DEEP LEARNING-BASED TRAFFIC MANAGEMENT ”, 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT) | 979-8-3503-7024-9/24/\$31.00 ©2024 IEEE | DOI: 10.1109/ICCCNT61001.2024.10724627

[23] Naman Jain, Rachit Parwanda, Anamika Chauhan “Real-Time Smart Traffic Control and Simulation: An Approach for Urban Congestion Management”, 2023 IEEE IAS Global Conference on Emerging Technologies (GlobConET) | 979-8-3503-3179-0/23/\$31.00 ©2023 IEEE | DOI: 10.1109/GlobConET56651.2023.10150057

[24] Svyatoslav A. Seliverstov, Arseny M. Sazanov , Yaroslav A. Seliverstov, Elena N. Benderskaya , Kirill V. Nikitin “Development of the Intelligent Traffic Management System Architecture”, 2021 XXIV International Conference on Soft Computing and Measurements (SCM) | 978-1-6654-3974-9/21/\$31.00 ©2021 IEEE | DOI: 10.1109/SCM52931.2021.9507125

[25] Farhain Sultana, Ritika Bhardwaj, Rohan N Kumar , Ruchir Gaur, Sahana P. Shankar, Aryan Bharadwaj “Smart Traffic Management System for Efficient Mobility and Emergency Response”, 2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS) | 979-8-3503-5968-8/24/\$31.00 ©2024 IEEE | DOI: 10.1109/ICKECS61492.2024.10616800

